

```

1 # Import required libraries
2
3 import pandas as pd
4 import numpy as np
5
6 from sklearn.preprocessing import LabelEncoder
7 from sklearn.model_selection import train_test_split
8 from sklearn.linear_model import LogisticRegression
9
10 from sklearn.metrics import accuracy_score
11 from sklearn.metrics import precision_score
12 from sklearn.metrics import recall_score
13 from sklearn.metrics import f1_score
14 from sklearn.metrics import classification_report

```

```

1 # Create employee dataset
2
3 data = {
4     "Age": [25, 30, 35, 28, 40, 32, 45, 29, 31, 38],
5
6     "Salary": [30000, 50000, 60000, 35000, 80000,
7               55000, 90000, 40000, 52000, 75000],
8
9     "Department": [
10         "IT", "HR", "Sales", "IT", "HR",
11         "Sales", "IT", "HR", "Sales", "IT"
12     ],
13
14     "YearsAtCompany": [
15         1, 5, 8, 2, 10,
16         6, 12, 3, 5, 9
17     ],
18
19     "JobSatisfaction": [
20         2, 4, 5, 2, 5,
21         4, 5, 3, 4, 5
22     ],
23
24     "Overtime": [
25         "Yes", "No", "No", "Yes", "No",
26         "Yes", "No", "Yes", "No", "No"
27     ],
28
29     "WorkLifeBalance": [
30         2, 4, 5, 2, 5,
31         3, 5, 2, 4, 5
32     ],
33
34     "Attrition": [
35         "Yes", "No", "No", "Yes", "No",
36         "Yes", "No", "Yes", "No", "No"
37     ]
38 }
39

```

```

40 df = pd.DataFrame(data)
41
42 print(df)

```

	Age	Salary	Department	YearsAtCompany	JobSatisfaction	Overtime	\
0	25	30000	IT	1	2	Yes	
1	30	50000	HR	5	4	No	
2	35	60000	Sales	8	5	No	
3	28	35000	IT	2	2	Yes	
4	40	80000	HR	10	5	No	
5	32	55000	Sales	6	4	Yes	
6	45	90000	IT	12	5	No	
7	29	40000	HR	3	3	Yes	
8	31	52000	Sales	5	4	No	
9	38	75000	IT	9	5	No	

	WorkLifeBalance	Attrition
0	2	Yes
1	4	No
2	5	No
3	2	Yes
4	5	No
5	3	Yes
6	5	No
7	2	Yes
8	4	No
9	5	No

```

1 # TASK - 1
2 # Display dataset information
3
4 print("Dataset Information:\n")
5
6 print(df.info())
7
8 print("\nFirst Five Rows:\n")
9
10 print(df.head())

```

Dataset Information:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Age                   10 non-null    int64
1   Salary                10 non-null    int64
2   Department            10 non-null    object
3   YearsAtCompany        10 non-null    int64
4   JobSatisfaction        10 non-null    int64
5   Overtime               10 non-null    object
6   WorkLifeBalance        10 non-null    int64
7   Attrition              10 non-null    object
dtypes: int64(5), object(3)
memory usage: 772.0+ bytes
None

```

First Five Rows:

	Age	Salary	Department	YearsAtCompany	JobSatisfaction	Overtime	\
0	25	30000	IT	1	2	Yes	
1	30	50000	HR	5	4	No	
2	35	60000	Sales	8	5	No	
3	28	35000	IT	2	2	Yes	
4	40	80000	HR	10	5	No	

	WorkLifeBalance	Attrition
0	2	Yes
1	4	No
2	5	No
3	2	Yes
4	5	No

```

1 # TASK - 2
2 # Create encoder
3
4 encoder = LabelEncoder()
5
6 # Encode Department
7
8 df["Department"] = encoder.fit_transform(
9     df["Department"]
10 )
11
12 # Encode Overtime
13
14 df["Overtime"] = encoder.fit_transform(
15     df["Overtime"]
16 )
17
18 # Encode Attrition
19
20 df["Attrition"] = encoder.fit_transform(
21     df["Attrition"]
22 )
23
24 print(df)

```

	Age	Salary	Department	YearsAtCompany	JobSatisfaction	Overtime	\
0	25	30000	1	1	2	1	
1	30	50000	0	5	4	0	
2	35	60000	2	8	5	0	
3	28	35000	1	2	2	1	
4	40	80000	0	10	5	0	
5	32	55000	2	6	4	1	
6	45	90000	1	12	5	0	
7	29	40000	0	3	3	1	
8	31	52000	2	5	4	0	
9	38	75000	1	9	5	0	

	WorkLifeBalance	Attrition
0	2	1
1	4	0
2	5	0
3	2	1
4	5	0
5	3	1
6	5	0

7	2	1
8	4	0
9	5	0

```
1 # TASK - 3
2 # Check missing values
3
4 print("Missing Values:\n")
5
6 print(df.isnull().sum())
```

Missing Values:

Age	0
Salary	0
Department	0
YearsAtCompany	0
JobSatisfaction	0
Overtime	0
WorkLifeBalance	0
Attrition	0

dtype: int64

```
1 # Features
2
3 X = df.drop(
4     "Attrition",
5     axis=1
6 )
7
8 # Target
9
10 y = df["Attrition"]
11
12 # Split data
13
14 X_train,X_test,y_train,y_test = train_test_split(
15     X,
16     y,
17     test_size=0.2,
18     random_state=42
19 )
20
21 print("Training Data Shape")
22
23 print(X_train.shape)
24
25 print("Testing Data Shape")
26
27 print(X_test.shape)
```

Training Data Shape
(8, 7)
Testing Data Shape
(2, 7)

```

1 # TASK - 4
2 # Create model
3
4 model = LogisticRegression()
5
6 # Train model
7
8 model.fit(
9     X_train,
10    y_train
11 )
12
13 print("Model Trained Successfully")

```

Model Trained Successfully

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465:
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regress

```
n_iter_i = _check_optimize_result(
```

```

1 # TASK - 5
2 # Prediction
3
4 y_pred = model.predict(
5     X_test
6 )
7
8 # Accuracy
9
10 accuracy = accuracy_score(
11     y_test,
12     y_pred
13 )
14
15 # Precision
16
17 precision = precision_score(
18     y_test,
19     y_pred
20 )
21
22 # Recall
23
24 recall = recall_score(
25     y_test,
26     y_pred
27 )
28
29 # F1 Score
30
31 f1 = f1_score(
32     y_test,

```

```

33     y_pred
34 )
35
36 print("Accuracy:",accuracy)
37
38 print("Precision:",precision)
39
40 print("Recall:",recall)
41
42 print("F1 Score:",f1)
43
44 print("\nClassification Report\n")
45
46 print(
47     classification_report(
48         y_test,
49         y_pred
50     )
51 )

```

```

Accuracy: 0.0
Precision: 0.0
Recall: 0.0
F1 Score: 0.0

```

Classification Report

	precision	recall	f1-score	support
0	0.00	0.00	0.00	2.0
1	0.00	0.00	0.00	0.0
accuracy			0.00	2.0
macro avg	0.00	0.00	0.00	2.0
weighted avg	0.00	0.00	0.00	2.0

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:156
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:156
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:156
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:156
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:156
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:156
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:156
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

```

1 # TASK - 6
2 # Feature importance
3
4 importance = pd.DataFrame({
5
6     "Feature":X.columns,
7

```

```

8     "Coefficient":
9     model.coef_[0]
10 })
11
12 print(
13     importance
14 )
15
16 # Highest impact
17
18 most_important = importance.loc[
19
20     importance[
21         "Coefficient"
22     ].abs().idxmax()
23 ]
24
25 print(
26     "\nMost Important Factor:"
27 )
28
29 print(
30     most_important[
31         "Feature"
32     ]
33 )

```

	Feature	Coefficient
0	Age	0.506490
1	Salary	-0.000215
2	Department	0.172357
3	YearsAtCompany	-0.457322
4	JobSatisfaction	-0.101110
5	Overtime	0.365786
6	WorkLifeBalance	-0.463031

Most Important Factor:
Age

```

1 # TASK - 7
2 # New employee data
3
4 new_employee = [[
5     27,      # Age
6     40000,   # Salary
7     1,       # Department IT
8     2,       # Years
9     2,       # Satisfaction
10    1,       # Overtime Yes
11    2        # WorkLifeBalance
12 ]]
13
14 prediction = model.predict(new_employee)
15
16 print("Prediction:")
17
18 if prediction[0] == 1:

```

```
19     print("Employee may leave company")
20 else:
21     print("Employee stays")
```

```
Prediction:
Employee may leave company
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: User
warnings.warn(
```

```
Employee may leave company
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: User
  warnings.warn(
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: User
warnings.warn(
```